SEVA SADAN'S

R.K. TALREJA COLLEGE OF ARTS, SCIENCE & COMMERCE

Department of Computer Science

A Project Report on

Railway Reservation System

In fulfilment of the requirement for the completion of the Internal Assessment for

Data Structure

Submitted by

Name of Student: - Harshada Kene

Roll No: - 2542015

S.Y.BSc. Information Technology (NEP 2020)

SEM III A.Y. 2025 - 26

1

Table of Contents

Sr. No.	Particulars	Page No.
1.	Introduction	3
2.	Definition	3
3	Objectives and Features	3
4.	Code.	5
5.	Output with screenshot	11
6.	Real life Applications.	14

🚆 Railway Reservation System – Project Report

📌 1. Introduction

The Railway Reservation System is a console-based C++ program designed to automate and streamline the process of booking, canceling, and managing train tickets. This project is based on fundamental data structures and object-oriented programming principles, using a **linked list** to manage passenger records dynamically. It demonstrates how a simple software system can effectively handle the core operations of a ticket reservation system.

📘 2. Definition

The **Railway Reservation System** is a computerized application that facilitates the management of railway ticket bookings. It allows passengers to book seats, cancel reservations, search ticket details, and check seat availability. In this C++ implementation, passenger data is stored in a **singly linked list**, allowing dynamic memory allocation and efficient record handling without the use of databases.

🎯 3. Objectives and Features

◈ Objectives:

- To design a user-friendly and efficient reservation system.
- To implement core operations like booking, canceling, and searching tickets.
- To manage available seats and display current bookings.

- To apply linked list data structure for real-time memory allocation.

◈ Key Features:

1. **Book Ticket:**
   - Allows users to input their details (name, age, train name) and books a seat.
   - Automatically assigns Ticket ID and Seat Number.
2. **Cancel Ticket:**
   - Enables passengers to cancel their booking using their Ticket ID.
   - Updates the seat count accordingly.
3. **Search Ticket:**
   - Lets the user find ticket details using the Ticket ID.
   - Displays all relevant passenger and journey details.
4. **Display All Bookings:**
   - Shows a list of all passengers who have booked tickets.
   - Displays ticket ID, name, age, train name, and seat number.
5. **Check Available Seats:**
   - Displays the number of available seats out of total (100 in this case).
6. **Seat Limitation Logic:**
   - The system does not allow booking if all 100 seats are already reserved.
7. **Dynamic Memory Allocation:**
   - Uses a linked list to store passenger data, enabling efficient memory use and flexibility.

🖊 4. Code

```
#include <iostream>
#include <string>
using namespace std;
const int TOTAL_SEATS = 100; // total seats available

// Structure for a passenger
struct Passenger {
    int ticketID;
    string name;
    int age;
    string trainName;
    int seatNo;
    Passenger* next;
};

// Head pointer for linked list
Passenger* head = NULL;
int nextTicketID = 1;
int bookedSeats = 0;
// Function to book a ticket
void bookTicket() {
    if (bookedSeats >= TOTAL_SEATS) {
        cout << "\n ❌ No seats available!\n";
        return;
    }
```

```
    Passenger* newPassenger = new Passenger;
    newPassenger->ticketID = nextTicketID++;
    cout << "\nEnter Passenger Name: ";
    cin.ignore();
    getline(cin, newPassenger->name);
    cout << "Enter Age: ";
    cin >> newPassenger->age;
    cout << "Enter Train Name: ";
    cin.ignore();
    getline(cin, newPassenger->trainName);
    newPassenger->seatNo = bookedSeats + 1;
    newPassenger->next = NULL;

  // Insert at end
    if (head == NULL) {
        head = newPassenger;
    } else {
        Passenger* temp = head;
        while (temp->next != NULL) temp = temp->next;
        temp->next = newPassenger;
    }
    bookedSeats++;
    cout << "\n✅ Ticket booked successfully!";
    cout << "\nTicket ID: " << newPassenger->ticketID
         << " | Seat No: " << newPassenger->seatNo << "\n";
}
```

```
// Function to cancel a ticket
void cancelTicket() {
    if (head == NULL) {
        cout << "\n❌ No bookings to cancel.\n";
        return;
    }
    int id;
    cout << "\nEnter Ticket ID to cancel: ";
    cin >> id;
    Passenger* temp = head;
    Passenger* prev = NULL;
    while (temp != NULL && temp->ticketID != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "\n❌ Ticket ID not found.\n";
        return;
    }
    if (prev == NULL) {
        head = temp->next;
    } else {
        prev->next = temp->next;
    }
    cout << "\n✅ Ticket ID " << id << " cancelled successfully.\n";
```

```
    delete temp;
    bookedSeats--;
}

// Function to search for a ticket
void searchTicket() {
    if (head == NULL) {
        cout << "\n❌ No bookings available.\n";
        return;
    }
    int id;
    cout << "\nEnter Ticket ID to search: ";
    cin >> id;
    Passenger* temp = head;
    while (temp != NULL) {
        if (temp->ticketID == id) {
            cout << "\n📄 Ticket Found:";
            cout << "\nTicket ID: " << temp->ticketID;
            cout << "\nName: " << temp->name;
            cout << "\nAge: " << temp->age;
            cout << "\nTrain: " << temp->trainName;
            cout << "\nSeat No: " << temp->seatNo << "\n";
            return;
        }
        temp = temp->next;
    }
```

```
    cout << "\n❌ Ticket not found.\n";
}

// Function to display all bookings
void displayAllBookings() {
    if (head == NULL) {
        cout << "\n❌ No bookings available.\n";
        return;
    }
    cout << "\n📋 All Bookings:\n";
    Passenger* temp = head;
    while (temp != NULL) {
        cout << "Ticket ID: " << temp->ticketID
            << " | Name: " << temp->name
            << " | Age: " << temp->age
            << " | Train: " << temp->trainName
            << " | Seat No: " << temp->seatNo << "\n";
        temp = temp->next;
    }
}

// Function to check available seats
void checkAvailableSeats() {
    cout << "\n🚆 Available Seats: " << (TOTAL_SEATS - bookedSeats)
         << " / " << TOTAL_SEATS << "\n";
}
```

```
int main() {
    int choice;
    do {
        cout << "\n\n====== Railway Reservation System ======";
        cout << "\n1. Book Ticket";
        cout << "\n2. Cancel Ticket";
        cout << "\n3. Search Ticket";
        cout << "\n4. Display All Bookings";
        cout << "\n5. Check Available Seats";
        cout << "\n6. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: bookTicket(); break;
            case 2: cancelTicket(); break;
            case 3: searchTicket(); break;
            case 4: displayAllBookings(); break;
            case 5: checkAvailableSeats(); break;
            case 6: cout << "\nExiting... Thank you!\n";
            break;
            default: cout << "\n ❌ Invalid choice! Try again.\n";
        }
    } while (choice != 6);
    return 0;  }
```

📤 5. Output with screenshot

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice:
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 1

Enter Passenger Name: Harshada Kene
Enter Age: 18
Enter Train Name: Rajdhani Express

✅ Ticket booked successfully!
Ticket ID: 1 | Seat No: 1
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 2

Enter Ticket ID to cancel: 3

✅ Ticket ID 3 cancelled successfully.
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 3

Enter Ticket ID to search: 1

🎫 Ticket Found:
Ticket ID: 1
Name: Harshada Kene
Age: 18
Train: Rajdhani Express
Seat No: 1
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 4

📋 All Bookings:
Ticket ID: 1 | Name: Harshada Kene | Age: 18 | Train: Rajdhani Express
    | Seat No: 1
Ticket ID: 2 | Name: Simran Khan | Age: 19 | Train: Vande Bharat
    Express | Seat No: 2
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 5

📊 Available Seats: 98 / 100
```

```
====== Railway Reservation System ======
1. Book Ticket
2. Cancel Ticket
3. Search Ticket
4. Display All Bookings
5. Check Available Seats
6. Exit
Enter your choice: 6

Exiting... Thank you!
```

🌍 6. Real-Life Applications

A Railway Reservation System like this forms the backbone of various large-scale, real-world systems. While this implementation is simple and educational, its core logic mirrors the operations found in real railway systems, such as:

- **Indian Railways Booking System (IRCTC)**
- **Amtrak (USA) Online Booking**
- **European Rail Ticketing Portals**
- **Rail Reservation Kiosks at Stations**

✨ Real-world Applications Include:

- **Online Ticket Booking Platforms**
- **Railway Station Booking Counters**
- **Mobile Reservation Apps**
- **Automated Customer Service Kiosks**
- **Passenger Management Systems**

Such systems are integral to managing high volumes of passengers, reducing manual errors, and providing quick and efficient service across vast networks.